

ChatGPT / Codex Recovery

Codex Perspective

Incident findings, verified recovery procedure, and operational lessons

Prepared July 30, 2026

Final status: Successful recovery. The Windows installation was stabilized, every exported conversation was verified, the project workspace was preserved, and the original local Codex sidebar registry was restored through a dry-run-tested offline merge.

Executive summary

This incident exposed four distinct recovery layers: the Windows app package, cloud-backed ChatGPT conversations, the filesystem workspace, and Codex's local sidebar/project registry. Treating those layers as interchangeable produced misleading signals. Recovery succeeded only after each layer was preserved, inspected, and validated independently.

ChatGPT guided the lengthy preservation, Windows repair, reinstall, and evidence-gathering process. Codex then used the recovered export and local artifacts to verify all conversations, reconcile the project tree, identify the correct pre-crash registry, and complete an offline merge with rollback protection and database integrity checks.

Codex perspective: what completed the recovery

- Conversation presence and sidebar visibility were separate questions. All 148 exported conversations were accessible even when the sidebar showed only a recent subset.
- The initially inspected active .codex snapshot was created after reinstall and had an empty thread table. It was not the authoritative pre-crash source.
- The decisive artifact was .codex.backup-20260728-012149: 261 thread records, 24 saved workspace roots, 21 original project-order entries, 10 pinned threads, plus matching session files.
- The safe operation was a merge, not a transplant. Current settings and new tasks were retained; old rows and missing session files were added by ID.
- A disposable dry run proved the merge first. The real run created a rollback snapshot, copied only missing files, merged global-state maps and arrays, and required SQLite integrity = OK.

Verified recovery outcomes

Recovery layer	Verified outcome
Conversation verification	148 of 148 accessible; zero failures
Local Codex restoration	261 preserved threads merged; 263 total afterward
Workspace registry	24 saved roots; 22 combined project-order entries
Pinned and indexed state	10 pinned threads; 137 session-index entries
Database validation	SQLite integrity check: OK
Post-recovery snapshot	262,617 files; 12.529 GB; zero mismatches or failures

PART I

Issues, Findings, and Lessons Learned

1. Primary app failure

- The Microsoft Store ChatGPT/Codex Windows package repeatedly failed with AppModel Runtime event 208 and error 0x8000001A.
- The visible dialog stated: “The application is exiting and cannot service this request.”
- The app also entered update/repair loops, including error 0x80073D02 when package files were in use.
- The failed package was OpenAI.Codex version 26.721.4979.0.
- Resets, reinstalls, profile resets, AppContainer resets, Store/AppX repair, lifecycle attempts, DISM, SFC, and reboots did not permanently correct that build.

2. Excessive recovery cost

- The incident consumed nearly 48 hours from the last response near the crash on Monday at 11:53 PM to restoration late Wednesday night.
- The recovery required preservation of more than 262,000 files, two independent verified backup copies, a data export, Windows integrity scans, a replacement app build, conversation verification, and a custom offline sidebar merge.
- The app exposed poor recovery ergonomics: internal state, visible sidebar indexing, cloud-backed conversations, project roots, and the actual filesystem workspace behaved as separate recovery layers.

3. WindowsApps permission risk

- A broad ownership/permission command was accidentally started against C:\Program Files\WindowsApps.
- The WindowsApps root owner temporarily became Everyone. The root was repaired to NT SERVICE\TrustedInstaller.
- A recursive ownership operation reported 0 successful files and approximately 89,000 failures, which strongly suggested child ownership was not broadly changed.
- The correct lesson is to avoid /T, /R, or blanket ACL resets on the entire WindowsApps tree. Repair only the specific object that was changed.

4. Orphaned package residue

- An old package directory could remain under WindowsApps even after uninstall. Such residue is not, by itself, evidence of disk or Windows corruption.

`C:\Program Files\WindowsApps\OpenAI.Codex_26.721.4979.0_x64__2p2nqsd0c76g0`

- The folder resisted deletion. Broad WindowsApps permission changes would have been riskier than leaving the orphaned app residue in place.

5. Integrity-check interpretation

- DISM CheckHealth and ScanHealth both reported no component-store corruption.
- CHKDSK completed in 2.47 hours and reported no file-system problems and 0 KB in bad sectors.
- SFC initially repaired protected files, but the recurring repair was a OneDrive shortcut in the Default user profile, not broad system damage.

`C:\Users\Default\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\OneDrive.lnk`

- A later SFC scan reported no integrity violations. There is no evidence that ChatGPT/Codex caused the SFC repair; only temporal association was observed.

6. Time and ETA errors during troubleshooting

- CHKDSK percentage and ETA values were volatile and should not have been treated as reliable completion predictions.
- Several earlier responses assigned exact timestamps without a grounded time source. Future reports should use only console timestamps, log timestamps, or an actual time tool.
- Progress should be judged by changing counters and log activity, not by unstable ETAs.

7. Store metadata was insufficient

- Winget showed Store product ID 9PLM9XGG6VKS and refreshed product text but reported Version: Unknown.
- Listing metadata changes did not prove that a new executable build existed.
- The decisive check was Get-AppxPackage after installation. It showed replacement build 26.721.11231.0, which was newer than the failed 26.721.4979.0 build.

8. Local .codex snapshots were easy to misidentify

The active post-reinstall .codex directory contained sizeable SQLite WAL files, which initially suggested recoverable task history might be present.

Read-only schema inspection showed that the active post-reinstall snapshot was empty: threads = 0, goals = 0, memories = 0. That result applied only to that snapshot, not to every preserved .codex backup.

- The active snapshot's logs database contained 228 rows and remained useful for diagnosis, but its empty thread table could not restore the former sidebar.
- The active post-reinstall config.toml initially contained only a trust entry for C:\Windows\System32, likely inherited from an elevated shell working directory.
- The inspected active skills and plugins trees contained system-managed or cached content; portable recovery value was limited.
- A second preserved source, .codex.backup-20260728-012149, was later identified as the true pre-crash registry. It contained 261 threads, 24 saved workspace roots, 21 original project-order entries, 10 pinned threads, and matching sessions. The distinction between these snapshots was decisive.

9. Conversation recovery model

- Codex tasks/chats were cloud-backed and visible through the recovered account/project state, even though the old local threads table was empty.
- The export was a preservation and comparison source, not a direct import package.
- All 148 exported conversations were accessible in Codex, including all 3 archived conversations. One additional newer conversation existed after the export cutoff.
- Database replacement was therefore unnecessary and inappropriate for conversation recovery.

10. Sidebar and project-index separation

- The underlying conversations and workspace were recovered before the sidebar visibly reflected them.
- The sidebar was a separate local UI/index problem, not proof that the chats or projects were missing.
- A dry-run-tested offline merge restored 261 thread index entries, 24 workspace roots, 22 project-order entries, and 10 pinned threads while preserving current recovery tasks.
- The restore script created a rollback snapshot before modifying the local index.

11. Backup performance and path-depth issues

- Copy-Item failed on deeply nested temporary plugin paths and later appeared to hang while walking generated dependency trees.
- The problematic paths were under .codex\.tmp and generated plugin structures. These were not necessary portable recovery data.
- Robocopy with explicit exclusions and process/log monitoring was more reliable for the large project tree.
- Do not restart a long-running backup merely because the command window times out. First check whether the process is active and whether the log is advancing.

12. What worked well

- Two independent, verified preservation copies before destructive changes.
- A separate account data export extracted under C:\CodeProjects\Export.
- Windows integrity checks performed before reinstalling the app.
- Verifying the actual installed AppX version rather than trusting Store metadata.
- Read-only database inspection instead of immediate transplantation.
- Comparing all exported conversations against live Codex visibility.
- Creating a post-recovery project snapshot after conversation verification.
- Dry-running the sidebar merge and creating rollback data before the real change.

13. Current evidence retained

Location	Purpose
C:\Users\Owner\Desktop\ChatGPT-Codex-Preservation-20260728-101536	Primary preservation set
C:\backupCodeProjects\ChatGPT-Codex-Preservation-20260728-101536	Second verified preservation copy
C:\CodeProjects\Export	Export, recovery inventory, logs, and restore package
C:\Users\Owner\Desktop\codex-old-db-analysis	Read-only old database analysis copy
C:\Users\Owner\Desktop\CodeProjects-post-recovery-20260729	Post-recovery workspace snapshot

14. Final assessment

The replacement build is functional, the workspace is preserved, all exported Codex conversations are verified, and the sidebar/project index has been restored. The incident demonstrated that the Windows app's lifecycle and recovery behavior were the principal failure points, while Codex's account-backed conversation data and recovery capabilities ultimately enabled a complete restoration.

PART II

Recovery Steps and Final Procedure

1. Preserve the workspace before changing the app

1. Treat the live project tree as the source of truth:

`C:\CodeProjects`

2. Create and retain two independent preservation copies. The verified copies used were:

`C:\Users\Owner\Desktop\ChatGPT-Codex-Preservation-20260728-101536`

`C:\backupCodeProjects\ChatGPT-Codex-Preservation-20260728-101536`

3. Verify the project copy with Robocopy and accept only a result with zero mismatches and zero failed files. The verified preservation set contained 262,592 files and 13,410,709,947 bytes. The temporary `.tmp.driveupload` directory was intentionally excluded.
4. Request a ChatGPT data export and keep both the downloaded archive and the extracted copy. The extracted export was stored at:

`C:\CodeProjects\Export`

2. Remove the failed installation without touching project data

5. Uninstall the Microsoft Store package and verify that Winget no longer reports it as installed.
6. Uninstall the global Codex CLI package separately:
`npm.cmd uninstall -g @openai/codex`
7. Remove obvious Codex setup leftovers and stale backup/cache directories, but do not delete the live workspace or either preservation copy.
8. Do not recursively reset ownership or ACLs across the entire WindowsApps tree. Only repair the WindowsApps root if its ownership was changed.

3. Repair and verify Windows

9. Restore the owner of the WindowsApps root to TrustedInstaller and confirm that no explicit Everyone permission remains on that root.
10. Run DISM health checks from Administrator PowerShell:

`DISM /Online /Cleanup-Image /CheckHealth`

`DISM /Online /Cleanup-Image /ScanHealth`

11. Run System File Checker from Administrator PowerShell:

`sfc.exe /scannow`

12. Run an online disk scan and let it finish even if the ETA fluctuates:

`chkdsk C: /scan`

13. Restart Windows and repeat SFC until the final result is:

Windows Resource Protection did not find any integrity violations.

14. The completed CHKDSK result reported no file-system problems and 0 KB in bad sectors. DISM reported no component-store corruption.

4. Identify the recurring SFC repair

15. Review only the newest [SR] entries in CBS.log instead of reading the entire file.
16. The recurring item was the Default-profile OneDrive shortcut:
`C:\Users\Default\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\OneDrive.lnk`
17. After SFC restored that shortcut, the next scan completed cleanly. Do not attribute this repair to ChatGPT or Codex without evidence.

5. Install the replacement Microsoft Store build

18. Install the Store product with Winget:
`winget install --id 9PLM9XGG6VKS --source msstore --exact --accept-source-agreements --accept-package-agreements`
19. Before launching, verify the installed AppX package version:
`Get-AppxPackage OpenAI.Codex | Select-Object Name, Version, PackageFullName, InstallLocation`
20. Confirm that the build is newer than the failed version. The replacement build was 26.721.11231.0; the failed build was 26.721.4979.0.
21. Launch the app and verify basic stability: successful load, clean sign-in/sign-out, and no immediate lifecycle error.

6. Protect the new local Codex state

22. Close Codex fully before copying local state. Use the tray Quit command and confirm no ChatGPT/Codex process remains. End only the specific app process if necessary.
23. Do not overwrite the new .codex directory with the old one. The directory contains version-sensitive databases, caches, installation identifiers, runtime paths, and credentials.
24. Create a small portable backup of the new configuration, excluding temporary and generated trees. The successful backup was:

`C:\Users\Owner\Desktop\codex-portable-backup-20260729-192515`

7. Inspect the old .codex snapshot safely

25. Distinguish the empty active post-reinstall snapshot from the true pre-crash registry. The successful source was:
`C:\Users\Owner\Desktop\ChatGPT-Codex-Preservation-20260728-101536\UserProfile-Codex\.codex.backup-20260728-012149`
26. Inspect only top-level names first. Do not recursively traverse .tmp or plugin caches unless needed.
27. Do not copy auth.json, installation_id, sandbox files, caches, temp directories, model caches, version.json, or old SQLite files into the live installation.
28. The active post-reinstall config.toml only trusted C:\Windows\System32 and was not useful as a restoration source. Preserve the newer live configuration rather than replacing it wholesale.

29. Copy the old SQLite database sets to a separate analysis directory and inspect them read-only. The analysis copy used was:

`C:\Users\Owner\Desktop\codex-old-db-analysis`

30. The active post-reinstall state database contained zero thread rows, zero goals, and zero memories. The separate pre-crash backup database contained 261 thread rows and was the correct source for the offline sidebar merge.

8. Verify cloud-backed Codex chats and projects

31. Keep the working installation intact and compare the export with chats visible in Codex.
32. The export contained 148 conversations: 145 active and 3 archived, dated June 11 through July 28, 2026.
33. Verify every exported conversation against Codex. Final result: all 148 were accessible, all 98 older conversations were retrieved, all 3 archived conversations were retrieved, zero failed verification, and one newer chat existed beyond the export cutoff.
34. Save the recovery inventory:

`C:\CodeProjects\Export\RECOVERY-INVENTORY.md`
`C:\CodeProjects\Export\recovery-inventory.csv`
35. Do not replace the live Codex database when the cloud-backed conversations are already available.

9. Create a post-recovery project snapshot

36. Create a separate post-recovery snapshot without altering the original preservation set or export.

`C:\Users\Owner\Desktop\CodeProjects-post-recovery-20260729`
37. Monitor the existing Robocopy process and log instead of starting a second copy. The completed snapshot preserved 262,617 files and 12.529 GB with zero mismatches and zero failed files.

10. Restore the Codex sidebar index offline

38. Prepare and dry-run an offline merge package while Codex remains open only for inspection.
39. The dry run projected 261 preserved threads, 24 workspace roots, 22 combined project-order entries, and 10 pinned threads. SQLite integrity was OK, and current recovery tasks were preserved.
40. Close Codex completely in Task Manager.
41. Run the prepared offline restore script:

`C:\CodeProjects\Export\RUN-SIDEBAR-RESTORE.cmd`
42. The completed merge added 261 thread rows and 4 spawn-edge rows, produced 263 total local threads and 137 session-index entries, retained 24 roots and 10 pins, and passed SQLite integrity validation.
43. Reopen Codex and verify the visible sidebar tree, project ordering, pinned threads, workspace roots, and recent tasks. The successful run logged completion at 11:11:54 PM on July 29, 2026.

11. Final verified state

- WindowsApps root owner restored to NT SERVICE\TrustedInstaller.

- No explicit Everyone ACL remained on the WindowsApps root.
- DISM CheckHealth and ScanHealth reported no corruption.
- CHKDSK reported no file-system problems and no bad sectors.
- Final SFC scan reported no integrity violations.
- Replacement app build 26.721.11231.0 launched and remained stable.
- All 148 exported Codex conversations were verified, including archived items.
- The workspace, project roots, thread index, pinned threads, and sidebar were restored.
- The original preservation set, export, old database analysis, and post-recovery snapshot were retained.

12. Codex-specific recovery safeguards

- Never conclude that all local history is absent after inspecting only one .codex snapshot. Enumerate sibling backup directories and compare database row counts, timestamps, global-state keys, and session folders.
- Do not overwrite auth.json, installation_id, runtime caches, or the complete live .codex directory. Merge only the state required for recovery.
- Stop Codex completely before modifying SQLite or .codex-global-state.json. An open process can retain WAL state or rewrite the registry during shutdown.
- Create a rollback snapshot immediately before the offline merge, then validate total row counts and PRAGMA integrity_check before reopening Codex.
- Keep the original preservation sets immutable. Recovery scripts, inventories, logs, and generated reports belong in a separate working directory.

13. Final assessment

The recovery was successful because the process shifted from broad repair attempts to evidence-based layer isolation. The failed Windows app build was replaced; the operating system and disk were verified; 148 cloud-backed conversations were confirmed; 262,617 project files were preserved in the post-recovery snapshot; and the pre-crash local registry was merged without sacrificing current state. The strongest operational lesson is simple: preserve first, identify the authoritative snapshot, test the merge on a disposable copy, and require measurable verification at every layer.